

MEAN STACK

(PEIT-109)

ASSIGNMENT

GURU NANAK DEV ENGINEERING COLLEGE

SUBMITTED IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE OF

BACHELORS OF TECHNOLOGY

(INFORMATION TECHNOLOGY)



SUBMITTED BY:

SHARNJEET SINGH (2303059)

DEPARTMENT OF INFORMATION TECHNOLOGY

GURU NANAK DEV ENGINEERING COLLEGE LUDHIANA

(An Autonomous College Under UGC ACT)

1. Q1: Login Page Web Application Using MEAN Stack Authentication

1.1. Aim

To construct a simple login page web application that authenticates users using the complete MEAN stack — MongoDB, Express.js, Angular, and Node.js — with **bcryptjs** for password hashing and **JSON Web Tokens (JWT)** for stateless session management.

1.2. Introduction

Authentication is the process of verifying who a user is before granting access to protected resources. In this project (*The Elevated Navigator* — a Travel Itinerary Planner), the login system is built across all four layers of the MEAN stack:

- **MongoDB** stores user documents with bcrypt-hashed passwords.
- **Express.js** exposes POST `/api/auth/login` and POST `/api/auth/register` REST endpoints.
- **Angular** renders the login form, calls the API, stores the JWT, and guards protected routes.
- **Node.js** is the runtime that ties everything together.

1.3. Packages Used

Package	Layer	Purpose
bcryptjs	Backend	Hash passwords before saving to MongoDB
jsonwebtoken	Backend	Sign and verify JWT tokens
dotenv	Backend	Load JWT_SECRET from <code>.env</code>
mongoose	Backend	User schema and database queries
@angular/common	Frontend	HttpClient, isPlatformBrowser
rxjs	Frontend	Reactive Observable / tap operator

1.4. Backend — Auth Middleware (server/middleware/auth.js)

Listing 1: JWT helpers and protect middleware

```
1 const jwt      = require('jsonwebtoken');
2 const bcrypt   = require('bcryptjs');
3 const User     = require('../models/User');
4
5 const JWT_SECRET = process.env.JWT_SECRET || 'mean_mini_secret_key';
6
7 // Middleware: verify JWT on every protected route
8 const authenticate = async (req, res, next) => {
9   try {
```

```

10  const authHeader = req.headers['authorization'];
11  // Header format: "Bearer <token>"
12  const token = authHeader && authHeader.split(' ')[1];
13
14  if (!token)
15    return res.status(401).json({ message: 'Access token
      required' });
16
17  // Decode and verify the token using the secret key
18  const decoded = jwt.verify(token, JWT_SECRET);
19
20  // Fetch the full user from DB (exclude password field)
21  const user = await User.findById(decoded.id).select('-password
    ');
22
23  if (!user || !user.isActive)
24    return res.status(401).json({ message: 'Invalid or expired
      token' });
25
26  req.user = user; // attach user to request for downstream
    use
27  next();
28  } catch (error) {
29    return res.status(401).json({ message: 'Invalid or expired
      token' });
30  }
31  };
32
33  // Sign a new JWT valid for 7 days
34  const generateToken = (userId) =>
35    jwt.sign({ id: userId }, JWT_SECRET, { expiresIn: '7d' });
36
37  // Hash a plain-text password with bcrypt (salt rounds = 10)
38  const hashPassword = async (password) => {
39    const salt = await bcrypt.genSalt(10);
40    return bcrypt.hash(password, salt);
41  };
42
43  // Compare a plain-text password against a stored bcrypt hash
44  const comparePassword = async (plain, hashed) =>
45    bcrypt.compare(plain, hashed);
46
47  module.exports = { authenticate, generateToken, hashPassword,
    comparePassword };

```

1.5. Backend — Auth Routes (server/routes/authRoutes.js)

Listing 2: Register and Login endpoints

```

1  const express = require('express');
2  const User    = require('../models/User');

```

```

3  const { authenticate, generateToken,
4      hashPassword, comparePassword } = require('../middleware/
      auth');
5
6  const router = express.Router();
7
8  // POST /api/auth/register
9  router.post('/register', async (req, res) => {
10     try {
11         const { name, email, password } = req.body;
12
13         // Validate that all fields are present
14         if (!name || !email || !password)
15             return res.status(400).json({ success: false,
16                                     message: 'All fields are
17                                         required.' });
18
19         const cleanEmail = String(email).trim().toLowerCase();
20
21         // Prevent duplicate accounts
22         const existing = await User.findOne({ email: cleanEmail });
23         if (existing)
24             return res.status(400).json({ success: false,
25                                     message: 'User already exists
26                                         .' });
27
28         // Hash the password before storing      never save plain text
29         const hashed = await hashPassword(password);
30
31         const user = await User.create({
32             name: String(name).trim(),
33             email: cleanEmail,
34             password: hashed,
35             role: 'user',
36         });
37
38         // Issue a JWT immediately so the user is logged in after
39         // registering
40         const token = generateToken(user._id);
41
42         return res.status(201).json({
43             success: true,
44             token,
45             user: { id: user._id, name: user.name,
46                 email: user.email, role: user.role },
47         });
48     } catch (error) {
49         return res.status(500).json({ success: false,
50                                     message: 'Server error during
51                                         registration.' });
52     }
53 }

```

```

49 });
50
51 // POST /api/auth/login
52 router.post('/login', async (req, res) => {
53   try {
54     const { email, password } = req.body;
55
56     if (!email || !password)
57       return res.status(400).json({ success: false,
58                                     message: 'Email and password
59                                     are required.' });
60
61     // Look up the user by email
62     const user = await User.findOne({
63       email: String(email).trim().toLowerCase()
64     });
65
66     // Return the same generic error for both "not found" and "
67     // wrong password"
68     // to avoid leaking which emails are registered
69     if (!user || !user.isActive)
70       return res.status(401).json({ success: false,
71                                     message: 'Invalid email or
72                                     password.' });
73
74     // bcrypt.compare hashes the plain password and checks it
75     // against the stored hash
76     const isMatch = await comparePassword(password, user.password)
77     ;
78     if (!isMatch)
79       return res.status(401).json({ success: false,
80                                     message: 'Invalid email or
81                                     password.' });
82
83     // Credentials are valid      sign and return a JWT
84     const token = generateToken(user._id);
85
86     return res.json({
87       success: true,
88       message: 'Login successful',
89       token,
90       user: { id: user._id, name: user.name,
91               email: user.email, role: user.role },
92     });
93   } catch (error) {
94     return res.status(500).json({ success: false,
95                                   message: 'Server error during
96                                   login.' });
97   }
98 });
99

```

```
93 module.exports = router;
```

1.6. Frontend — Auth Service (frontend/src/app/services/auth.service.ts)

Listing 3: Angular service — stores JWT and exposes reactive state

```
1 @Injectable({ providedIn: 'root' })
2 export class AuthService {
3   private baseUrl = 'http://localhost:5000/api/auth';
4   // isPlatformBrowser prevents localStorage access during SSR
5   // prerendering
6   private isBrowser = isPlatformBrowser(inject(PLATFORM_ID));
7
8   // Angular signal      reactive current-user state used by navbar
9   // and guards
10  currentUser = signal<AuthUser | null>(this.loadUser());
11
12  // Read persisted user from localStorage on service init
13  private loadUser(): AuthUser | null {
14    if (!this.isBrowser) return null;
15    try { return JSON.parse(localStorage.getItem('user')) || 'null'; }
16    catch { return null; }
17  }
18
19  get token() { return this.isBrowser
20    ? localStorage.getItem('token') : null; }
21  get isLoggedIn() { return !!this.token; }
22
23  // Call the login endpoint; on success persist token + user
24  login(email: string, password: string): Observable<any> {
25    return this.http.post<any>(`${this.baseUrl}/login`,
26      { email, password }).pipe(
27      tap(res => {
28        if (res.success && this.isBrowser) {
29          localStorage.setItem('token', res.token);
30          localStorage.setItem('user', JSON.stringify(res.user));
31          this.currentUser.set(res.user); // update signal
32          instantly
33        }
34      })
35    );
36  }
37
38  // Clear token and redirect to login page
39  logout(): void {
40    if (this.isBrowser) {
41      localStorage.removeItem('token');
42      localStorage.removeItem('user');
43    }
44    this.currentUser.set(null);
45  }
46 }
```

```

42     this.router.navigate(['/login']);
43   }
44 }

```

1.7. Frontend — HTTP Interceptor (auth.interceptor.ts)

Listing 4: Attach Bearer token to every outgoing request automatically

```

1 export const authInterceptor: HttpInterceptorFn = (req, next) => {
2   const token = inject(AuthService).token;
3
4   if (token) {
5     // Clone the request and add the Authorization header
6     const cloned = req.clone({
7       setHeaders: { Authorization: `Bearer ${token}` },
8     });
9     return next(cloned); // forward the modified request
10  }
11
12  return next(req); // no token      send request as-is
13 };

```

1.8. Frontend — Route Guard (auth.guard.ts)

Listing 5: Block unauthenticated access to /dashboard and /admin

```

1 export const authGuard: CanActivateFn = () => {
2   const auth = inject(AuthService);
3   const router = inject(Router);
4
5   if (auth.isLoggedIn) return true; // token present      allow
6     navigation
7
8   router.navigate(['/login']); // no token      redirect to
9   login
10  return false;
11 };

```

1.9. Frontend — Login Component

Listing 6: login.component.ts — calls AuthService on form submit

```

1 export class LoginComponent {
2   email = ''; password = ''; error = ''; loading = false;
3
4   constructor(private auth: AuthService, private router: Router) {}
5
6   onSubmit() {

```

```

7      if (!this.email || !this.password) return;
8      this.loading = true;
9      this.error    = '';
10
11      this.auth.login(this.email, this.password).subscribe({
12          next: (res) => {
13              this.loading = false;
14              // Navigate to dashboard on successful login
15              if (res.success) this.router.navigate(['/dashboard']);
16              else this.error = res.message || 'Login failed';
17          },
18          error: (err) => {
19              this.loading = false;
20              // Show the server error message (e.g. "Invalid email or
21              // password")
22              this.error = err?.error?.message || 'Invalid credentials';
23          }
24      });
25  }

```

1.10. Authentication Flow Diagram

1. User fills email + password on Angular login form
2. Angular calls POST /api/auth/login via AuthService
3. Express finds user in MongoDB by email
4. bcrypt.compare(plain, hash) checks the password
5. On match: jwt.sign({id}, SECRET, {expiresIn: '7d'})
6. Token returned to Angular in JSON response
7. Angular stores token in localStorage
8. currentUser signal updated → navbar re-renders
9. HTTP Interceptor adds "Authorization: Bearer <token>" to every future request automatically
10. authGuard reads token to allow /dashboard access

1.11. Project Screenshots

Login Page:

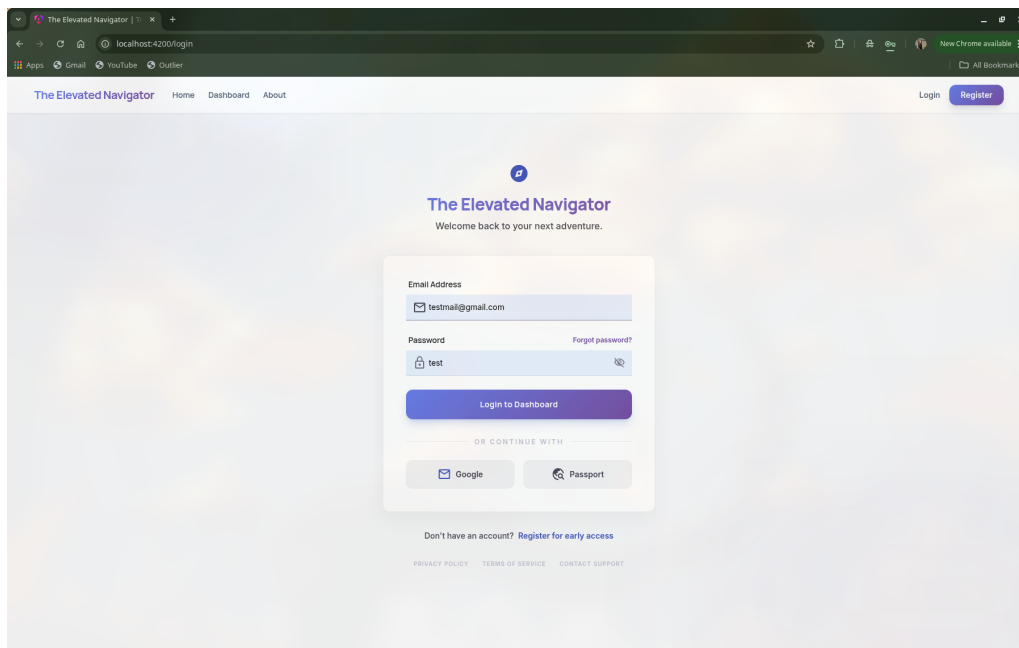


Figure 1: Login page of The Elevated Navigator

Dashboard (after successful login):

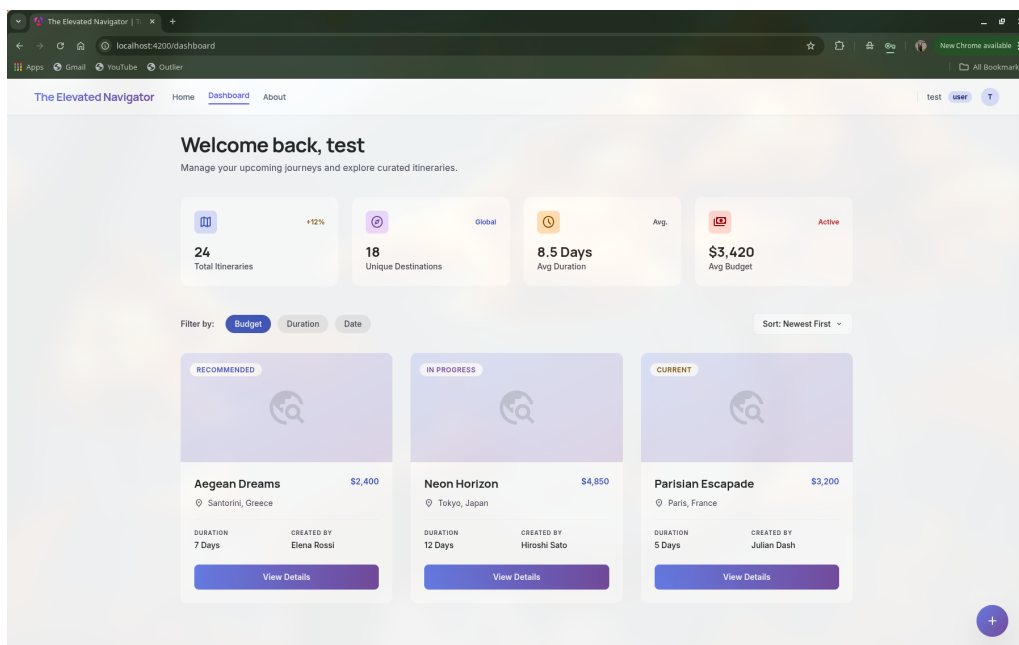


Figure 2: User dashboard — accessible only after authentication

2. Q2: Configure an Express Application with a Coding Example

2.1. Aim

To configure an Express.js application, explaining each part of the setup with in-code comments written in plain language.

2.2. Introduction to Express

Express is a minimal, unopinionated web framework for Node.js. It wraps Node's built-in `http` module and adds:

- A clean **routing API** (`app.get`, `app.post`, ...)
- A **middleware pipeline** — functions that run in sequence on every request
- Helpers for sending **JSON**, **HTML**, **files** and setting status codes

In this project the Express server lives in `server/server.js` and is the backbone of the entire backend API.

2.3. Step 1 — Install Dependencies

```
1 npm install express dotenv mongoose cors
```

2.4. Step 2 — Environment File (.env)

```
1 # Never commit this file      it holds secrets
2 MONGO_URI=mongodb://localhost:27017/travel_planner
3 PORT=5000
4 JWT_SECRET=your_super_secret_key
```

2.5. Step 3 — Database Connection (server/config/db.js)

Listing 7: Mongoose connection helper

```
1 const mongoose = require('mongoose');
2
3 // Async function so we can use await and catch errors cleanly
4 const connectDB = async () => {
5   const mongoUri = process.env.MONGO_URI;
6
7   // Stop the process immediately if the URI is missing from .env
8   if (!mongoUri) {
9     console.error('MONGO_URI is missing in .env');
10    process.exit(1);
11  }
```

```

12
13   try {
14       // mongoose.connect returns a promise      await it before
           continuing
15       await mongoose.connect(mongoUri, {
16           useNewUrlParser: true,    // use the new URL string parser
17           useUnifiedTopology: true,  // use the new server discovery
           engine
18       });
19       console.log('MongoDB connected');
20   } catch (error) {
21       // If the DB is unreachable, log the reason and exit
22       console.error('MongoDB connection error:', error.message);
23       process.exit(1);
24   }
25 };
26
27 module.exports = connectDB;

```

2.6. Step 4 — Main Server File (server/server.js)

Listing 8: Fully commented Express configuration

```

1 // Load environment variables from .env into process.env
2 // Must be called before any other code reads process.env
3 const dotenv = require('dotenv');
4 dotenv.config();
5
6 // Core Node.js module for working with file paths
7 const path = require('path');
8
9 // Express      the web framework
10 const express = require('express');
11
12 // Our custom DB connection helper
13 const connectDB = require('./config/db');
14
15 // Route handlers      each file exports an express.Router()
16 const authRoutes      = require('./routes/authRoutes');
17 const itineraryRoutes  = require('./routes/itineraryRoutes');
18 const userRoutes       = require('./routes/userRoutes');
19 const roleRequestRoutes = require('./routes/roleRequestRoutes');
20
21 //      Create the Express application instance
22
23
24 const app = express();
25
26
27 //      Connect to MongoDB

```

```

25 // This runs once at startup; if it fails the process exits (see
    db.js)
26 connectDB();
27
28 //          Global Middleware
29
30 // Parse incoming requests with a JSON body (e.g. from Angular
    HttpClient)
31 // Without this, req.body would be undefined for POST/PUT requests
32 app.use(express.json());
33
34 // Parse URL-encoded bodies (e.g. traditional HTML form
    submissions)
35 app.use(express.urlencoded({ extended: false }));
36
37 // Serve everything inside the /views folder as static files
38 // e.g. GET /login.html will return views/login.html directly
39 app.use(express.static(path.join(__dirname, '..', 'views')));
40
41 //          Page Routes (serve HTML files)
42
43 // Root URL          send the landing page
44 app.get('/', (req, res) => {
45     res.sendFile(path.join(__dirname, '..', 'views', 'index.html'));
46 });
47
48 // Login page
49 app.get('/login', (req, res) => {
50     res.sendFile(path.join(__dirname, '..', 'views', 'login.html'));
51 });
52
53 // Register page
54 app.get('/register', (req, res) => {
55     res.sendFile(path.join(__dirname, '..', 'views', 'register.html
        '));
56 });
57
58 // Dashboard page (Angular SPA handles auth on the client side)
59 app.get('/dashboard', (req, res) => {
60     res.sendFile(path.join(__dirname, '..', 'views', 'dashboard.html
        '));
61 });
62
63 //          API Routes
64
65 // app.use(prefix, router) mounts a router at a URL prefix.

```

```

65 // All routes defined inside authRoutes will be prefixed with /api
    // /auth
66 // e.g. router.post('/login') becomes POST /api/auth/login
67
68 app.use('/api/auth',          authRoutes);          // register,
    login, profile
69 app.use('/api/itinerary',      itineraryRoutes);    // CRUD +
    booking
70 app.use('/api/users',          userRoutes);          // user
    management
71 app.use('/api/role-requests',  roleRequestRoutes);  // admin role
    requests
72
73 //          Start the Server

74 const PORT = process.env.PORT || 5000;
75
76 // Special case: Vercel runs the app as a serverless function,
77 // so we export the app instead of calling listen()
78 if (process.env.VERCEL) {
79   module.exports = app;
80 } else {
81   // app.listen binds to the port and starts accepting connections
82   app.listen(PORT, () => {
83     console.log('Server running on port ${PORT}');
84   });
85 }

```

2.7. Step 5 — A Sample API Route (routes/itineraryRoutes.js)

Listing 9: Express Router with middleware chaining

```

1 const express    = require('express');
2 const Itinerary  = require('../models/Itinerary');
3 const { authenticate, authorize } = require('../middleware/auth');
4
5 // express.Router() creates a mini-app that handles a subset of
    routes
6 const router = express.Router();
7
8 // GET /api/itinerary
9 // authenticate runs first      if the token is invalid it sends
    401 and stops here
10 // If the token is valid, control passes to the async handler
11 router.get('/', authenticate, async (req, res) => {
12   try {
13     // Regular users only see active itineraries; admins see
        everything
14     const query = req.user.role === 'user' ? { isActive: true } :
        {};

```

```

15 // .populate replaces the createdBy ObjectId with the actual
16 // User document
17 const itineraries = await Itinerary.find(query)
18   .populate('createdBy', 'name email role')
19   .sort({ createdAt: -1 }); // newest first
20
21 return res.json(itineraries);
22 } catch (error) {
23   return res.status(500).json({ message: 'Server error.' });
24 }
25 });
26
27 // POST /api/itinerary
28 // Two middleware functions before the handler:
29 // 1. authenticate      verify JWT
30 // 2. authorize          check role is admin or superadmin
31 router.post('/', authenticate, authorize('admin', 'superadmin'),
32   async (req, res) => {
33     try {
34       const { title, destination, startDate,
35         endDate, duration, budget } = req.body;
36
37       // Basic validation before touching the database
38       if (!title || !destination || !startDate ||
39         !endDate || !duration || budget === undefined)
40         return res.status(400).json({ message: 'Fill all required
41           fields.' });
42
43       // Create the document; createdBy is taken from the verified
44       // JWT user
45       const itinerary = await Itinerary.create({
46         ...req.body,
47         createdBy: req.user._id,
48       });
49
50       return res.status(201).json(itinerary);
51     } catch (error) {
52       return res.status(500).json({ message: 'Server error.' });
53     }
54   });
55 // Export the router so server.js can mount it
56 module.exports = router;

```

2.8. How Middleware Chaining Works

When a request arrives at `POST /api/itinerary`, Express runs each function in the chain left-to-right:

```

Request arrives
↓
express.json() - parse the JSON body
↓
authenticate() - verify the JWT token
↓
authorize('admin',...) - check the user's role
↓
Route handler - create the itinerary in MongoDB
↓
res.json() - send the response back

```

Each middleware calls `next()` to pass control to the next function, or calls `res.status(...).json(...)` to short-circuit the chain and send an error response immediately.

2.9. Key Express Concepts Summary

Concept	What it does in this project
<code>express()</code>	Creates the application instance
<code>app.use()</code>	Registers global middleware or mounts a router
<code>express.json()</code>	Parses <code>application/json</code> request bodies
<code>express.static()</code>	Serves the <code>views/</code> folder as static files
<code>express.Router()</code>	Groups related routes into a separate file
<code>app.listen(PORT)</code>	Starts the HTTP server on the given port
<code>req.body</code>	Parsed request payload (needs <code>express.json()</code>)
<code>req.params.id</code>	URL parameter, e.g. <code>/itinerary/:id</code>
<code>req.user</code>	Custom property set by <code>authenticate</code> middleware
<code>res.json()</code>	Sends a JSON response with <code>Content-Type: application/json</code>
<code>res.status(404)</code>	Sets the HTTP status code before sending

GitHub Repository

The complete source code for this project(MINI PROJECT) (The Elevated Navigator — MEAN Stack Travel Itinerary Planner) is available at:

https://github.com/sharnjeet21/mean_mini

The repository contains:

- `server/` — Express + Node.js backend with JWT auth and Mongoose models
- `frontend/` — Angular 21 SPA with Tailwind CSS design system
- `views/` — Legacy static HTML pages
- `.env.example` — Environment variable template
- `README.md` — Setup and run instructions

Deployment

The project (The Elevated Navigator — MEAN Stack Travel Itinerary Planner) is deployed on render and can be visited at:

<https://mean-mini.onrender.com/>